

Теоретический материал по теме: Вложенные и каскадные условия

1. Введение в тему

- **Вложенные условия:**

Это условия, которые находятся внутри других условий. Они используются, когда необходимо проверить несколько взаимосвязанных условий.

Пример: сначала проверяется одно условие, и если оно истинно, проверяется следующее условие.

- **Каскадные условия:**

Это последовательная проверка нескольких условий. Программа идет сверху вниз по условиям, и как только одно из них выполнится, остальные не проверяются.

В Python каскадные условия достигаются с помощью конструкции if-elif-else.

2. Вложенные условия

Вложенные условия позволяют проверять дополнительное условие только после выполнения предыдущего. Это полезно, когда логика проверки требует пошаговых проверок.

Синтаксис вложенных условий:

```
if условие1:
    if условие2:
        блок кода1
    else:
        блок кода2
else:
    блок кода3
```

Пример программы:

```
age = 25
has_ticket = True
has_passport = False

if age >= 18:
    print("Взрослый")
    if has_ticket:
        print("Есть билет")
        if has_passport:
            print("Можно лететь за границу")
        else:
            print("Нужен загранпаспорт!")
    else:
        print("Билета нет -- не пустим")
```

```
else:  
    print("Несовершеннолетний")
```

Плюсы: понятно, можно много уровней

Минусы: быстро превращается в «пирамиду отступов» (рекомендуется не больше 3 уровней)

3. Каскадные условия

Каскадные условия полезны, когда нужно проверить несколько вариантов, и программа выбирает действие в зависимости от выполнения одного из условий. Если первое условие не выполнено, проверяются следующие.

Синтаксис каскадных условий:

```
if условие1:  
    блок кода1  
elif условие2:  
    блок кода2  
elif условие3:  
    блок кода3  
else:  
    блок кода4
```

Пример программы:

```
score = 77  
if score >= 90:  
    grade = "A"  
elif score >= 80:  
    grade = "B"  
elif score >= 70:  
    grade = "C"  
elif score >= 60:  
    grade = "D"  
else:  
    grade = "F"  
print(f"Оценка: {grade}") # Оценка: C  
Очень удобно для оценок, меню, состояния светофора и т.д.
```

4. Тернарный оператор

Тернарный оператор позволяет сократить запись условия в одну строку, что может быть полезно для компактных проверок.

Синтаксис:

```
значение_если_True if условие else значение_если_False
```

Пример:

```
age = 19
status = "Взрослый" if age >= 18 else "Ребёнок"
print(status) # Взрослый
# Можно и в несколько строк для читаемости
max_value = (a if a > b
             else b if b > c
             else c)
```

Важно: тернарный оператор — это выражение, а не инструкция. Его можно использовать там, где нужно значение (присваивание, аргумент функции, return и т.д.).

Когда что использовать:

- Вложенные if → когда проверки действительно независимы и могут выполняться одновременно
- if-elif-else → когда варианты взаимоисключающие
- Тернарный → когда нужно просто выбрать одно из двух значения (особенно в одну строку)

5. Сравнение нескольких значений одновременно

В Python можно проверять несколько значений в одном выражении, что делает код более читаемым.

Пример:

```
x = 5
if 1 <= x <= 10:
    print("x находится в диапазоне от 1 до 10.")
```

6. Использование булевых значений

В Python булевы значения True и False могут использоваться напрямую в условиях, что упрощает логику программы.

Пример:

```
is_active = True
if is_active:
    print("Аккаунт активен.")
```

7. Комбинация нескольких логических операторов

Для создания сложных условий можно комбинировать операторы `and`, `or`, `not` и использовать скобки для приоритета выполнения.

Пример:

```
age = 25
has_ticket = True
is_vip = False
if (age >= 18 and has_ticket) or is_vip:
    print("Добро пожаловать!")
```

8. Условные выражения с None

В Python значение `None` используется для обозначения отсутствия значения или пустой переменной.

Пример:

```
value = None
if value is None:
    print("Значение отсутствует.")
else:
    print("Значение:", value)
```